



CSCI 1101 Introduction to Computer Science

# Data Representation

# Overview

## Definitions

- Data
- Data Representation

## Number Systems

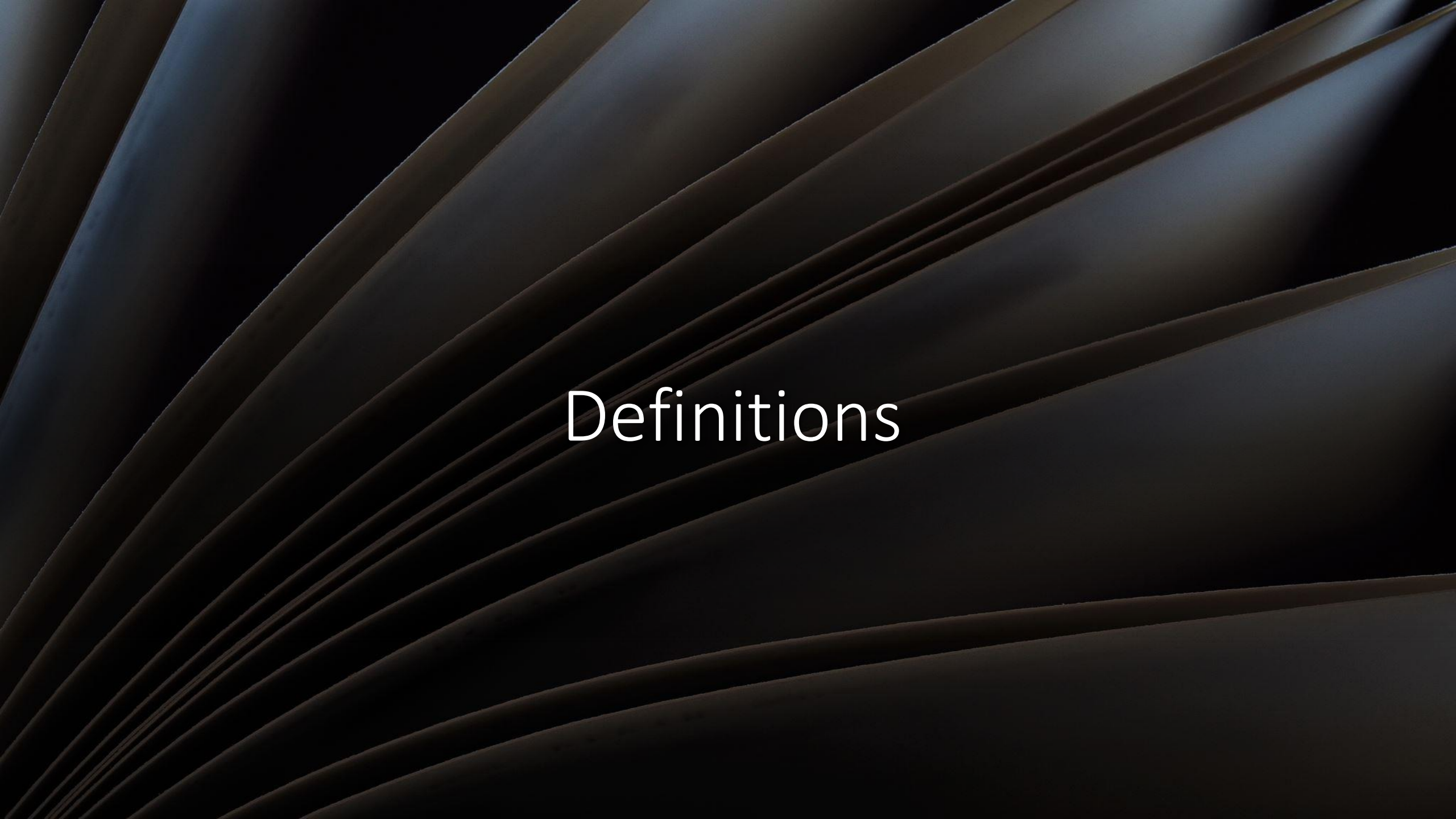
- Binary
- Decimal
- Hexadecimal

## Conversions

- From Decimal
- From Binary
- From Hexadecimal

## Computers & Data

- Representing Numbers
- Representing Text
- Representing Images
- Representing Sound
- Representing Video



# Definitions



# What is Data?

- **Data** refers to any collection of raw facts, figures, or information that can be processed, stored, and analyzed.
- Thus, data can be a:
  - Numbers
  - text
  - images
  - Audio
  - Video
  - other types of input



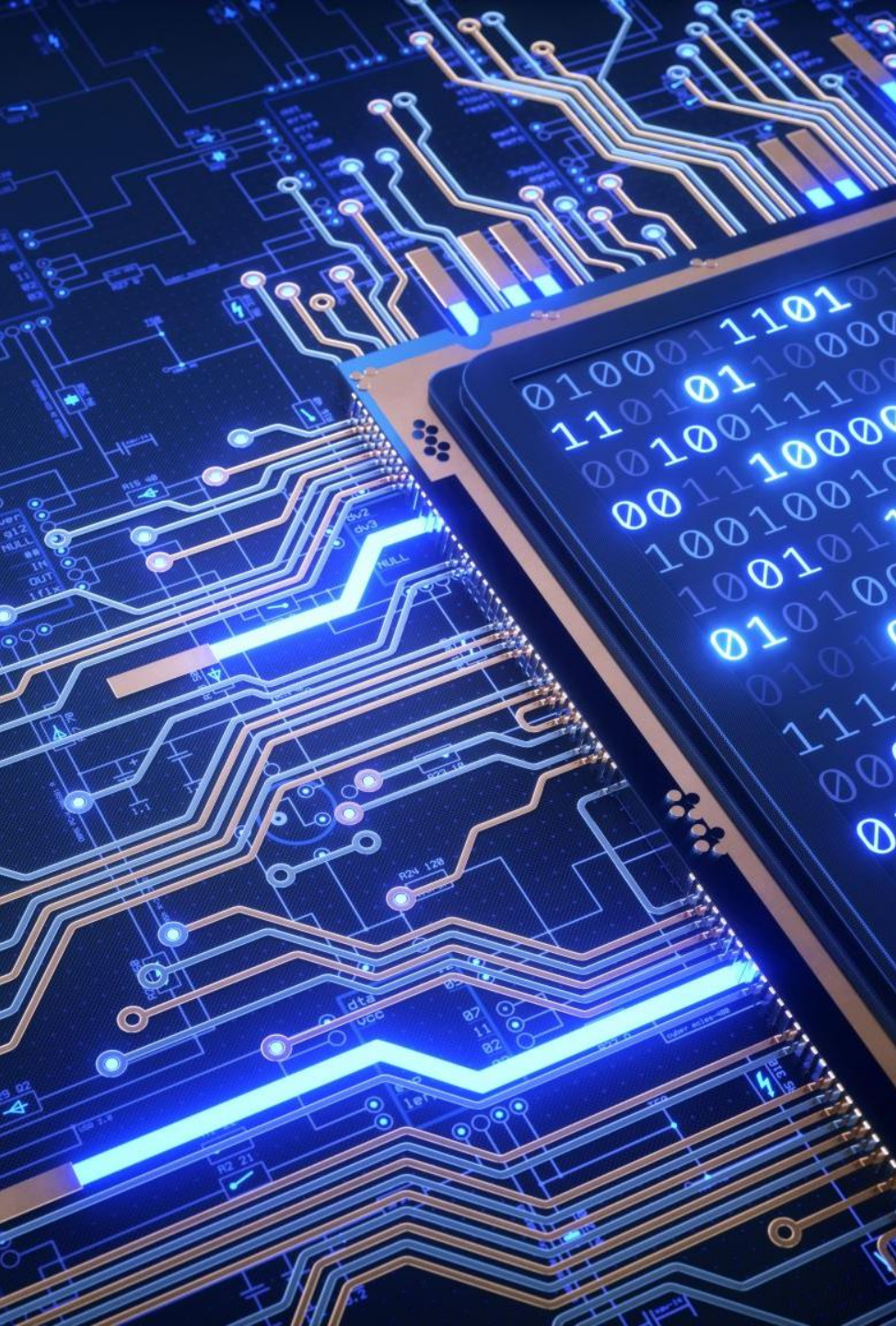
# Computers and Data

---

- **Data Representation** refers to the form in which data is stored, processed, and transmitted.
- Music, videos, games, and other information are the computer's data.
- Data is stored in digital (data expressed in **binary**) formats that can be handled by electronic circuitry.







# Why Binary?

- Computers operate with voltage through what amounts to microscopic switches (transistors) that are either “on” (1) or “off” (0).

Low Voltage = 0  
Hight Voltage = 1

0

ASCII codes  
represent data as  
0s and 1s

1

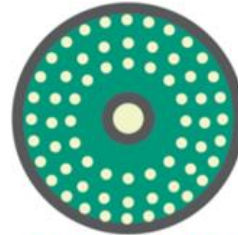
1 1 0 1 0 0 0 1 0 1 0 1 0 1 0 0 0 0



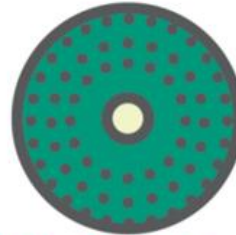
Circuit boards  
carry data as  
pulses of current



+5 +5 -2 +5 -2 -2 -2 +5 -2 +5 -2 +5 -2 +5 -2 -2 -2



CDs and DVDs  
store data as dark  
and light spots



.....

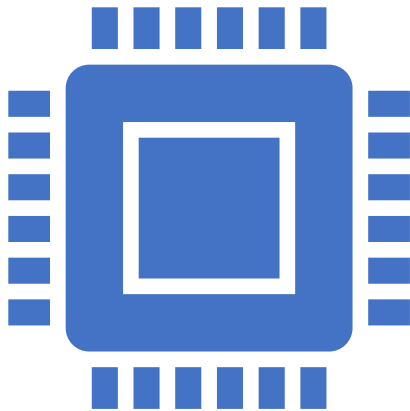


Disk drives store  
data as magnetized  
particles



+ + - + - - - + - + - + - - -

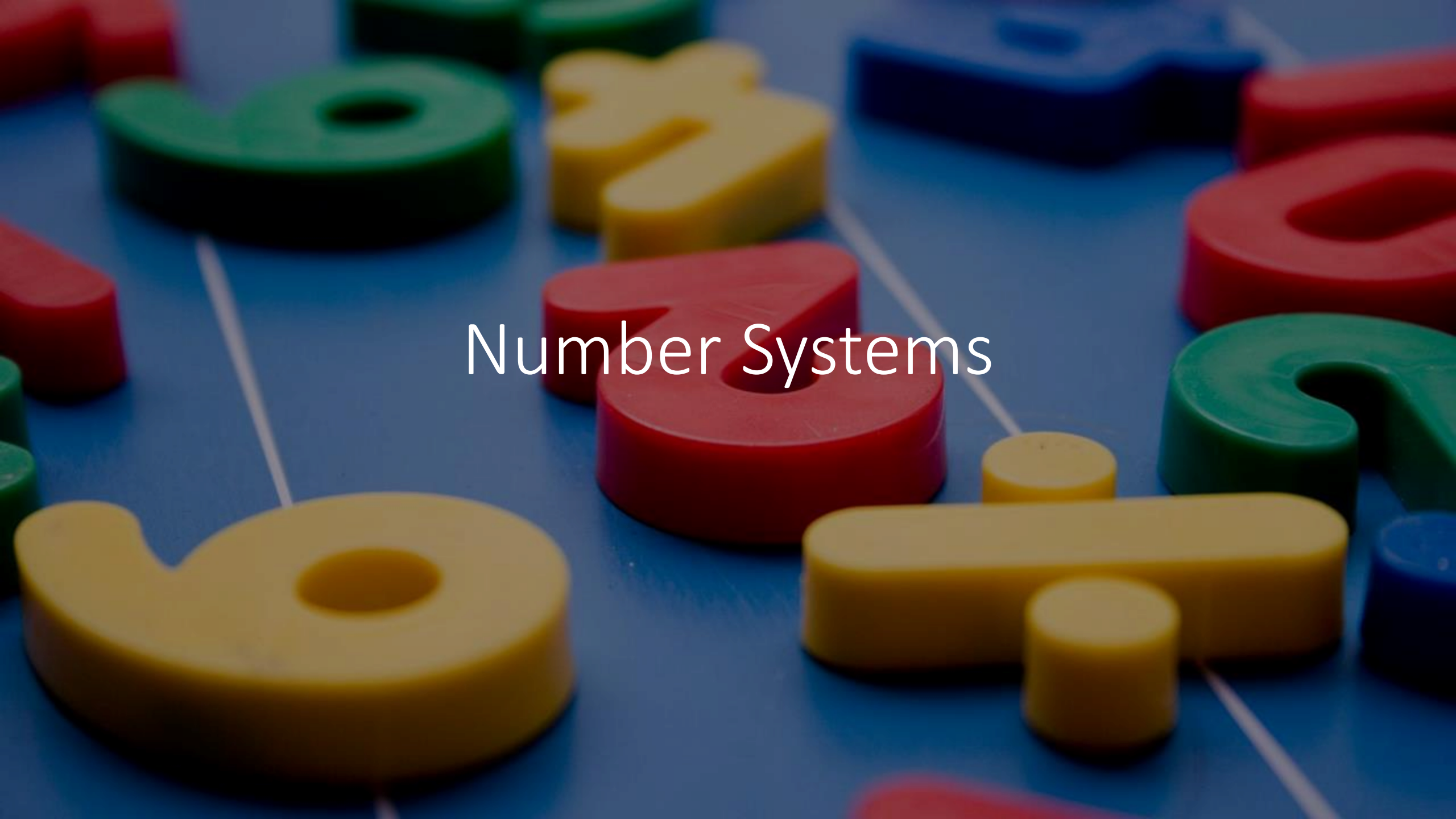
# Bits and Bytes



- Terminology related to bits and bytes is extensively used to describe storage capacity and network access speed.
  - Bits → binary digit
  - Byte → group of eight bits (abbreviated as an uppercase B)
- Have you ever heard phrases like “megabytes”, “terabytes”, or something similar?



| Unit      | Abbreviation | Approximate Size            |
|-----------|--------------|-----------------------------|
| Bit       | b            | Binary digit, single 1 or 0 |
| Nibble    | –            | 4 bits                      |
| Byte      | B            | 8 bits                      |
| Kilobyte  | KB           | 1,024 bytes or $10^3$       |
| Megabyte  | MB           | 1,024 KB or $10^6$          |
| Gigabyte  | GB           | 1,024 MB or $10^9$          |
| Terabyte  | TB           | 1,024 GB or $10^{12}$       |
| Petabyte  | PB           | 1,024 TB or $10^{15}$       |
| Exabyte   | EB           | 1,024 PB or $10^{18}$       |
| Zettabyte | ZB           | 1,024 EB or or $10^{21}$    |



# Number Systems



# Decimal Numbers

- Think about the numbers, you learned at school while growing up.
- They had 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- The number 1,248 is
$$1,000 + 200 + 40 + 8$$
- We can change the representation to:
$$(1 * 1,000) + (2 * 100) + (4 * 10) + (8 * 1)$$
- We can change it even further to:
$$(1 * 10^3) + (2 * 10^2) + (4 * 10^1) + (8 * 10^0)$$



# Deca

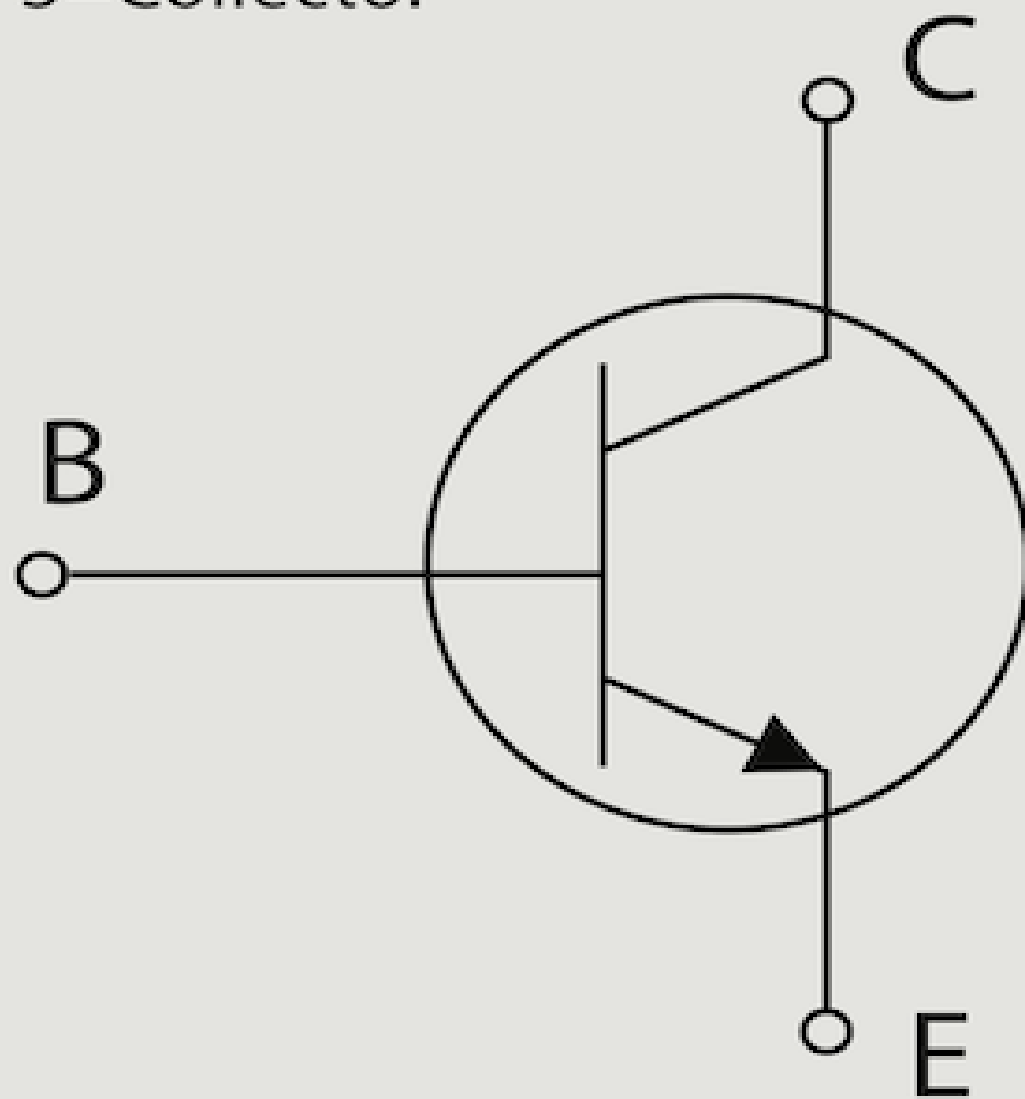
- Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Base 10

|        |        |        |        |
|--------|--------|--------|--------|
|        |        |        |        |
| $10^3$ | $10^2$ | $10^1$ | $10^0$ |
|        |        |        |        |



1 2 3

1=Emitter  
2=Base  
3=Collector

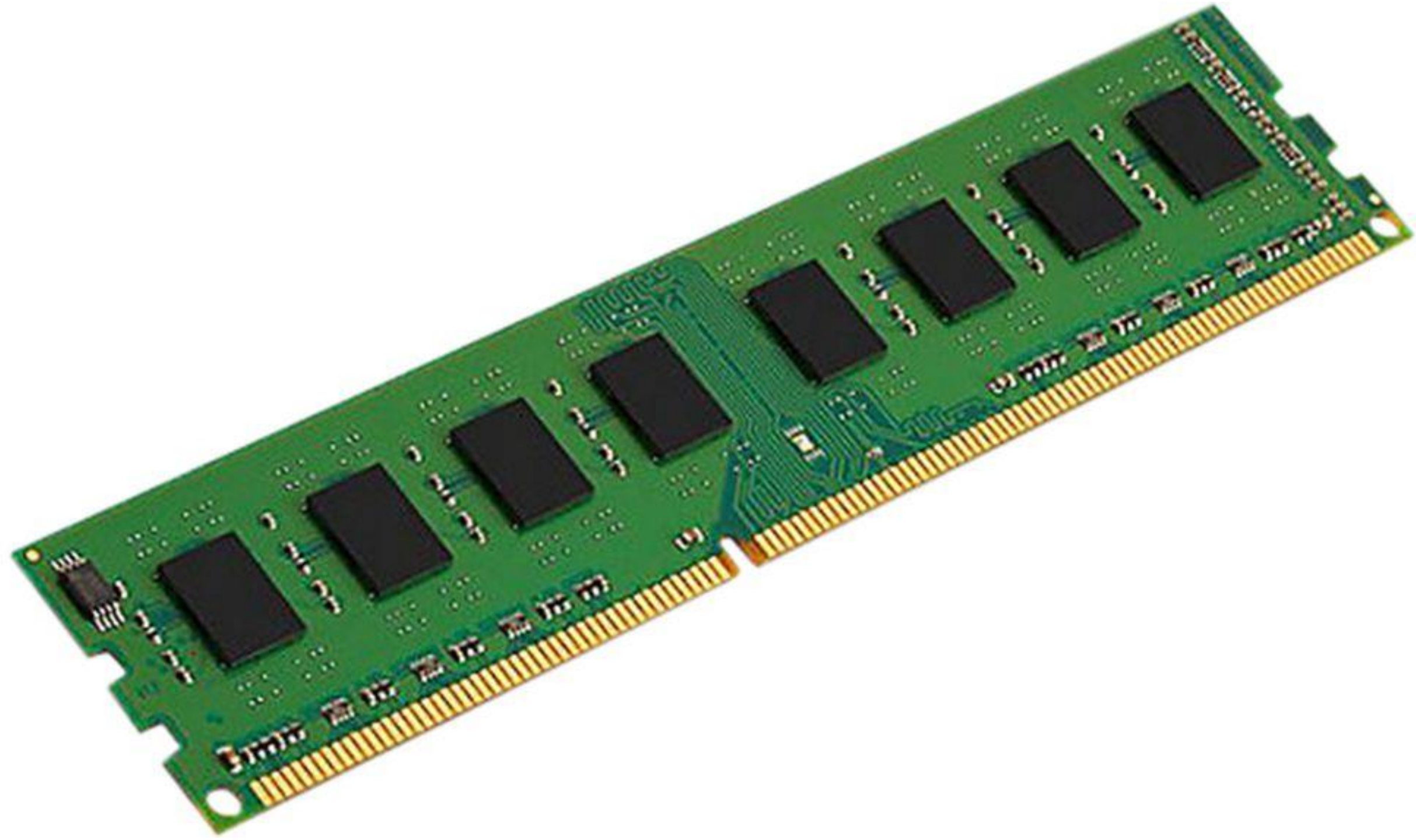


# Binary

- Digits: 0, 1
- Base 2
- Binary digit → *bit*

|       |       |       |       |
|-------|-------|-------|-------|
|       |       |       |       |
| $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|       |       |       |       |





# Hexadecimal

- Human-friendly way of grouping binary data
- Digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- Base 16
- Every hexadecimal digit corresponds to four binary bits.

1 = 0001

A = 1010

F = 1111

|        |        |        |        |
|--------|--------|--------|--------|
|        |        |        |        |
| $16^3$ | $16^2$ | $16^1$ | $16^0$ |
|        |        |        |        |

# Summary

| Decimal<br>(Base 10) | Binary<br>(Base 2) | Hexadecimal<br>(Base 16) |
|----------------------|--------------------|--------------------------|
| 0                    | 0000               | 0                        |
| 1                    | 0001               | 1                        |
| 2                    | 0010               | 2                        |
| 3                    | 0011               | 3                        |
| 4                    | 0100               | 4                        |
| 5                    | 0101               | 5                        |
| 6                    | 0110               | 6                        |
| 7                    | 0111               | 7                        |
| 8                    | 1000               | 8                        |
| 9                    | 1001               | 9                        |
| 10                   | 1010               | A                        |
| 11                   | 1011               | B                        |
| 12                   | 1100               | C                        |
| 13                   | 1101               | D                        |
| 14                   | 1110               | E                        |
| 15                   | 1111               | F                        |





# Addition and Subtraction



# Addition

$$\begin{array}{r} + \begin{array}{|c|c|c|c|} \hline 0 & 0 & 0 & 1 \\ \hline 0 & 0 & 0 & 0 \\ \hline \end{array} \\ \hline \begin{array}{|c|c|c|c|} \hline 0 & 0 & 0 & 1 \\ \hline \end{array} \end{array}$$

$$\begin{array}{r} \phantom{+} \begin{array}{|c|c|c|c|} \hline 0 & 0 & 0 & 1 \\ \hline 0 & 0 & 0 & 1 \\ \hline \end{array} \\ \phantom{+} 1 \\ \hline \begin{array}{|c|c|c|c|} \hline 0 & 0 & 1 & 0 \\ \hline \end{array} \end{array}$$

# Addition (Example)

$$0110_2 + 0011_2$$

|       |   |   |   |   |
|-------|---|---|---|---|
|       | 1 |   | 1 |   |
|       | 0 | 1 | 1 | 0 |
| +     | 0 | 0 | 1 | 1 |
| <hr/> |   |   |   |   |
|       | 1 | 0 | 0 | 1 |

Addition  
(Your turn)

$$00101_2 + 01111_2$$

|       |   |   |   |   |   |
|-------|---|---|---|---|---|
| +     | 0 | 0 | 1 | 0 | 1 |
|       | 0 | 1 | 1 | 1 | 1 |
| <hr/> |   |   |   |   |   |
|       | 1 | 0 | 1 | 0 | 0 |

# Subtraction

$$\begin{array}{r} 0001 \\ - 0000 \\ \hline 0001 \end{array}$$

$$\begin{array}{r} 0001 \\ - 0001 \\ \hline 0000 \end{array}$$



# Subtraction

## RULES:

1. We borrow from the column to the left.
2. If 0, we continue to the next column
3. If 1, this number becomes 0 and 2 is passed to the column to the right

-

|       |   |   |   |
|-------|---|---|---|
| 0     | 0 | 1 | 0 |
| 0     | 0 | 0 | 1 |
| <hr/> |   |   |   |
| 0     | 0 | 0 | 1 |



# Subtraction (Example)

$$0110_2 - 0011_2$$

|       |   |   |   |
|-------|---|---|---|
| 0     | 1 | 1 | 0 |
| 0     | 0 | 1 | 1 |
| <hr/> |   |   |   |
| 0     | 0 | 1 | 1 |

# Subtraction (Example)

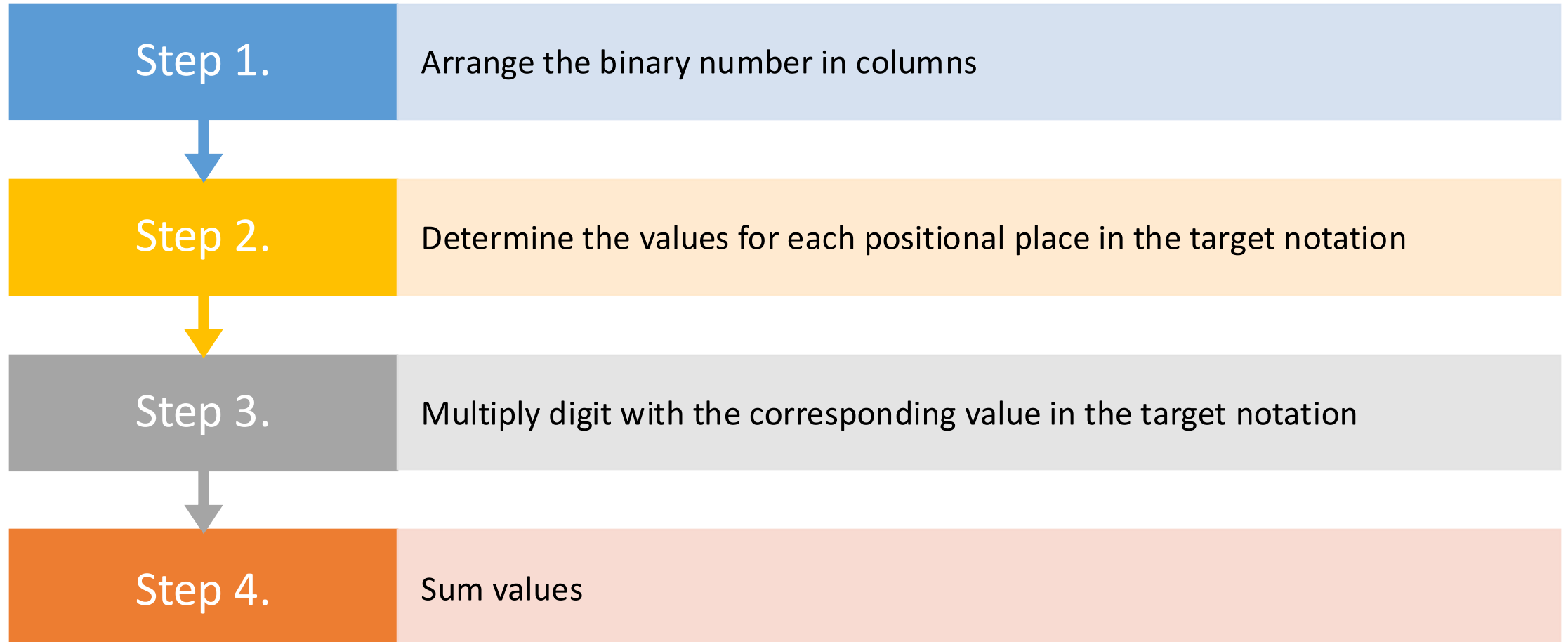
$$1100_2 - 0111_2$$

|       |   |   |   |
|-------|---|---|---|
| 1     | 1 | 0 | 0 |
| 0     | 1 | 1 | 1 |
| <hr/> |   |   |   |
| 0     | 1 | 0 | 1 |



Conversions

# Conversions to Decimal (Method 1)





# Example Binary to Dec

Start here →

$$10110011_2 = X_{10}$$

|                                     |                                    |                                    |                                    |                                   |                                   |                                   |                                   |
|-------------------------------------|------------------------------------|------------------------------------|------------------------------------|-----------------------------------|-----------------------------------|-----------------------------------|-----------------------------------|
| 1                                   | 0                                  | 1                                  | 1                                  | 0                                 | 0                                 | 1                                 | 1                                 |
| 128<br><small>2<sup>7</sup></small> | 64<br><small>2<sup>6</sup></small> | 32<br><small>2<sup>5</sup></small> | 16<br><small>2<sup>4</sup></small> | 8<br><small>2<sup>3</sup></small> | 4<br><small>2<sup>2</sup></small> | 2<br><small>2<sup>1</sup></small> | 1<br><small>2<sup>0</sup></small> |
| 128                                 |                                    | 32                                 | 16                                 |                                   |                                   | 2                                 | 1                                 |

$$10110011_2 = 179_{10}$$

sum

# Example Hex to Dec

Start here →

$$A2F7_{16} = X_{10}$$

|                                |                               |                              |                             |
|--------------------------------|-------------------------------|------------------------------|-----------------------------|
| A                              | 2                             | F                            | 7                           |
| 10                             | 2                             | 15                           | 7                           |
| 4096 <sub>16<sup>3</sup></sub> | 256 <sub>16<sup>2</sup></sub> | 16 <sub>16<sup>1</sup></sub> | 1 <sub>16<sup>0</sup></sub> |
| 40960                          | 512                           | 240                          | 7                           |

$$A2F7_2 = 41719_{10}$$

— sum —  
↓

# Converting to Decimal (Method 2)

Step 1.

Divide the value by the desired base number (B) and record the remainder.



Step 2.

As long as the quotient obtained is not zero, continue to divide the newest quotient by B and record the remainder.

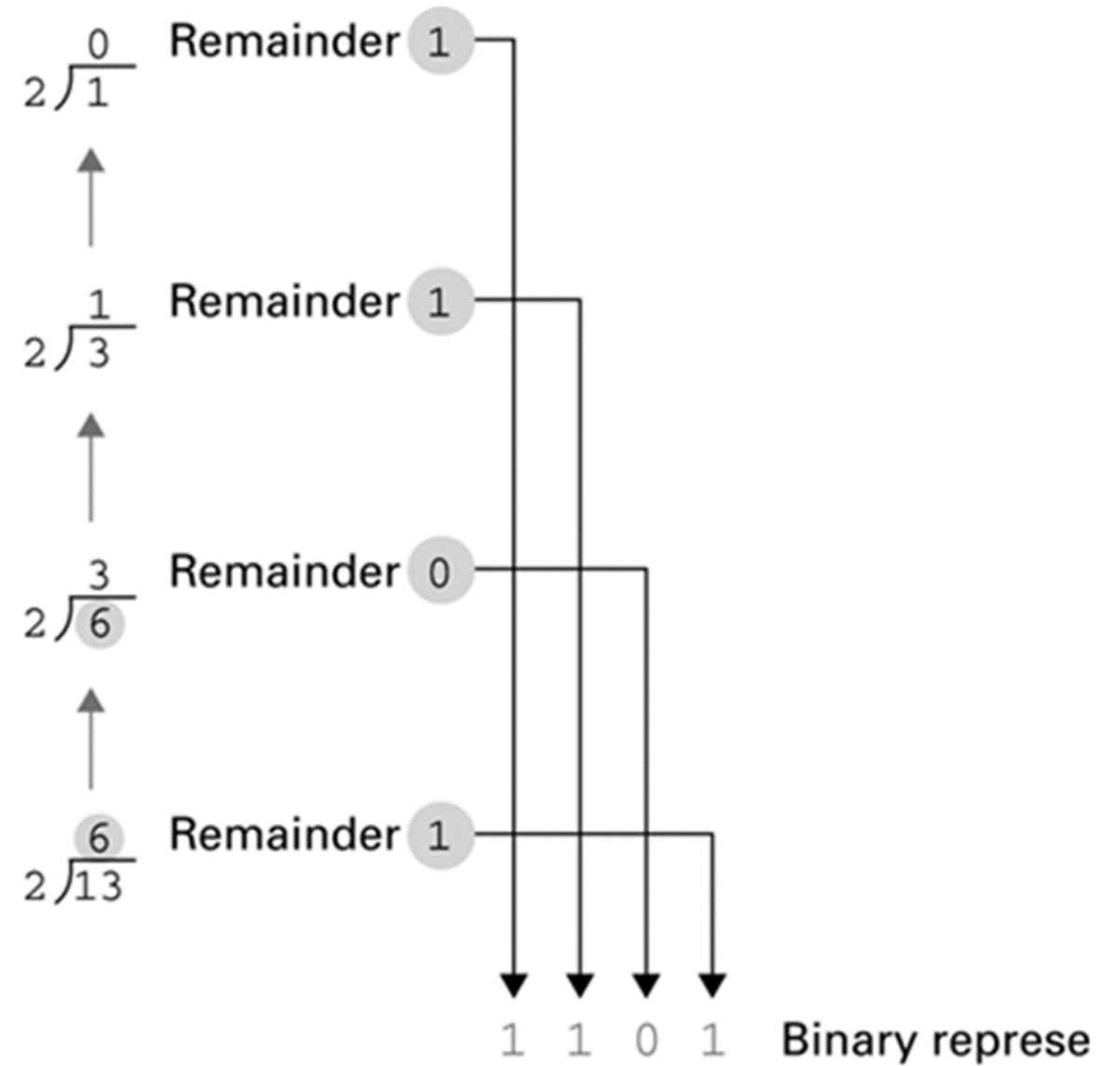


Step 3.

Now that a quotient of zero has been obtained, the new number system representation of the original value consists of the remainders listed from right to left in the order they were recorded.

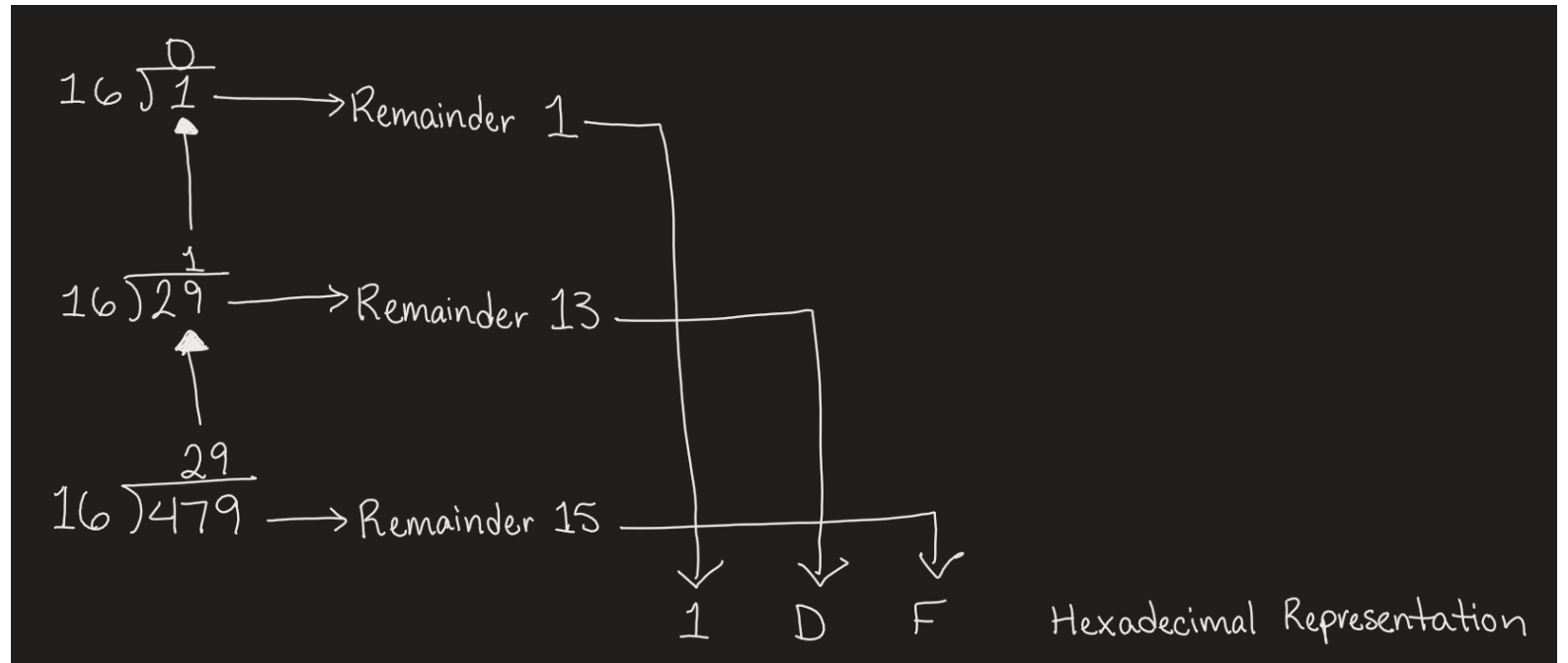
# Example 1-1

Start here →



# Example 1-2

Start here →





# Converting From Binary To Hexadecimal

**Step 1.** Mark groups of four bits (starting at the right end)

**Step 2.** Convert each group of bits into hex digits.

| Bit pattern | Hexadecimal representation |
|-------------|----------------------------|
| 0000        | 0                          |
| 0001        | 1                          |
| 0010        | 2                          |
| 0011        | 3                          |
| 0100        | 4                          |
| 0101        | 5                          |
| 0110        | 6                          |
| 0111        | 7                          |
| 1000        | 8                          |
| 1001        | 9                          |
| 1010        | A                          |
| 1011        | B                          |
| 1100        | C                          |
| 1101        | D                          |
| 1110        | E                          |
| 1111        | F                          |

10101011

1010

A

1011

B

Example 3

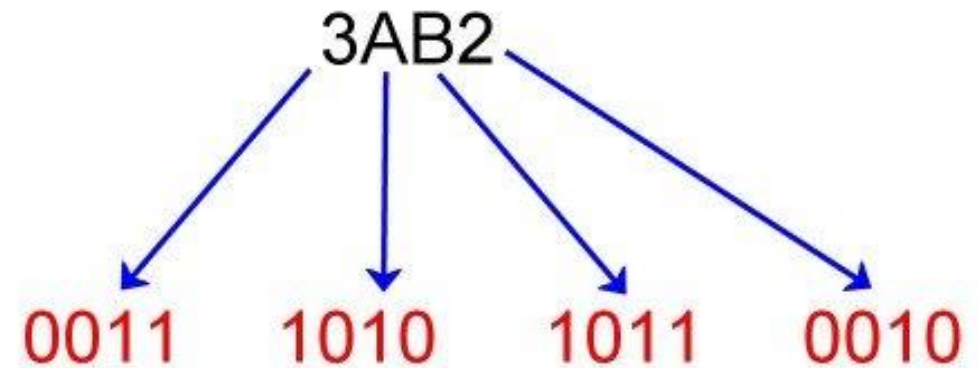
# Converting From Hexadecimal To Binary

**Step 1.** Convert each hex digit into binary from left to right

| Bit pattern | Hexadecimal representation |
|-------------|----------------------------|
| 0000        | 0                          |
| 0001        | 1                          |
| 0010        | 2                          |
| 0011        | 3                          |
| 0100        | 4                          |
| 0101        | 5                          |
| 0110        | 6                          |
| 0111        | 7                          |
| 1000        | 8                          |
| 1001        | 9                          |
| 1010        | A                          |
| 1011        | B                          |
| 1100        | C                          |
| 1101        | D                          |
| 1110        | E                          |
| 1111        | F                          |

## Example 4

### Converting Hex to Binary



$$3AB2_{16} = 11101010110010_2$$



The background is a dark, abstract digital scene. It features a grid of small, glowing blue and green dots, resembling a digital display or a network. Overlaid on this are several large, out-of-focus circular light spots in shades of orange, yellow, and blue, creating a bokeh effect. The overall impression is one of a high-tech, digital environment.

# Representing Numbers

# Representing Numeric Values

- Using the number system (binary, hexadecimal), whole numbers (0, 1, 2,...) can be represented
- What about integers and floating-point (real) numbers?
  - Modify binary system to represent these values as well



Representing  
Negative  
Values

---

Signed-Magnitude (SM)  
Notation

---

---

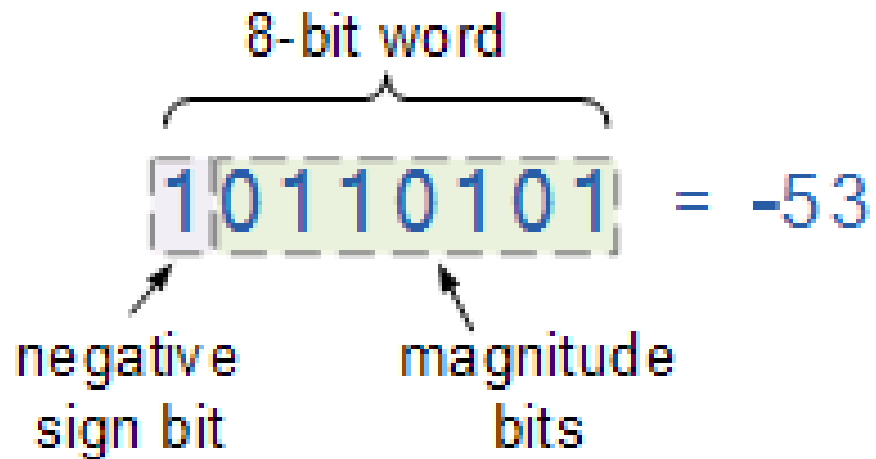
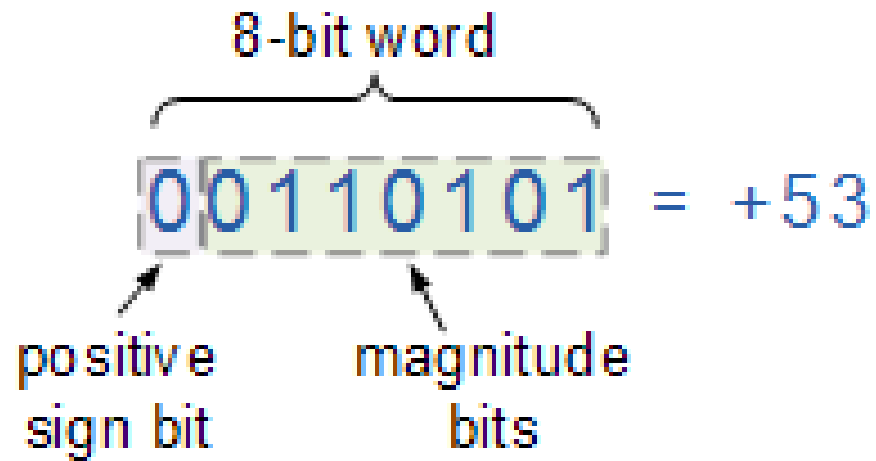
1's Complement

---

---

2's Complement

---



## Sign-Magnitude (SM) Notation

The leftmost bit (**most significant bit**) represents the sign:

- positive (0) or
- negative (1).

The rest of the binary digits are used to represent the magnitude of the binary number.

What is the problem with SM Notation?

There are two representations for 0, a positive and negative representation.

# 1's Complement

**Positive numbers** remain unchanged and start with a 0.  
**Negative numbers** are obtained by inverting the unsigned positive number (changing the 0 to 1 and 1 to 0).

## **Example: 8-bit representation of $\pm 12$**

+12: 0000 1100

-12: 1111 0011

What is the problem with 1's complement?

It also has two representations for 0.

+0: 0000

- 0: 1111

# 2's Complement

**Two's Complement** is the most widely used method for representing signed integers (both positive and negative) in binary.

Two's Complement = One's Complement + 1

**Example: 8-bit representation of  $\pm 12$**

+12: 0000 1100

-12: 1111 0011 // one's complement

---

1: 0000 0001 // plus one

-12: 1111 0100

There is a single representation for 0 with 2's Complement.

# Representing Real Numbers

- Consider weights and placement
  - Decimal System
    - Examples: 104.32, 0.99999, 357.0, and 3.14159
    - Weights to the right of the decimal are:  $10^{-1}$ ,  $10^{-2}$ ,  $10^{-3}$ , ...
  - Binary System
    - Examples: 10.101, 0.11101, 1011.1, and 11.111
    - Weights to the right of the decimal are:  $2^{-1}$ ,  $2^{-2}$ ,  $2^{-3}$ , ...

# Converting To Decimal

Step 1.

Multiply the digit by the quantity it represents ( $B^n$ ) and record the value. It is recommended to start from right end.



Step 2.

Repeat step 1 until no digits in the number given are left.



Step 3.

Add all the recorded values together to obtain the equivalent decimal value.



# Converting Binary Fractions to Decimal Fractions

- Same process – multiply each fractional bit by 2 (base) raised to the corresponding power and add the products together

- Example:

$$(0.101)_2 = ( \quad )_{10}$$

|                           |                           |                           |                            |                            |
|---------------------------|---------------------------|---------------------------|----------------------------|----------------------------|
| 1                         | 0                         | 1                         | 0                          | 0                          |
| $\frac{1}{2}$<br>$2^{-1}$ | $\frac{1}{4}$<br>$2^{-2}$ | $\frac{1}{8}$<br>$2^{-3}$ | $\frac{1}{16}$<br>$2^{-4}$ | $\frac{1}{32}$<br>$2^{-5}$ |

$$1 * 2^{-1} + 0 * 2^{-2} + 1 * 2^{-3}$$

This is equivalent to

$$1 * \frac{1}{2} + 0 * \frac{1}{4} + 1 * \frac{1}{8} = 0.625$$

Thus,  $(0.101)_2 = (0.625)_{10}$

# Real Numbers in Programming

---

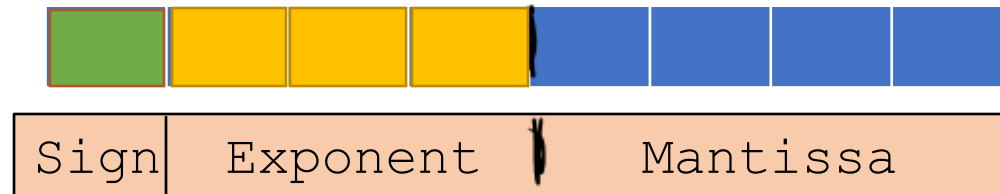
| Value  | Stored as                                  |
|--------|--------------------------------------------|
| Float  | sign bit, 8-bit exponent, 23-bit mantissa  |
| Double | sign bit, 11-bit exponent, 52-bit mantissa |

# Storing Real Numbers

- Storing real numbers in binary is done using a method called **floating-point representation**
- This breaks a number into three parts:
  - the **sign**,
  - the **mantissa** (or significand),
  - and the **exponent**.
- This format allows computers to store very large or very small real numbers efficiently.

# Storing Real Numbers

- **Sign:** A single bit: 0 for positive, 1 for negative
- **Mantissa:** In binary, the mantissa is normalized to always start with 1. followed by the fractional part. Only the fractional part is stored (since the leading 1. is implicit in the normalized form).
- **Exponent:** The exponent determines how many places to move the binary point, effectively scaling the mantissa. It is stored using biased representation.



# IEEE 754 Standard (single precision)

In IEEE 754 (the standard used in most systems for floating-point arithmetic), a 32-bit number is divided into:

1 bit for the sign

8 bits for the exponent, with a bias of 127

23 bits for the mantissa, storing the fractional part after normalization.

Example:  $(10.75)_{10} = (1010.11)_2$

Normalized number =  $1010.11$  becomes  $1.01011 \times 2^3$

Mantissa =  $01011000000000000000000$  (23 bits, fractional part only).

Exponent = Since the exponent is 3, we store  $3 + 127 = 130$  in binary as  $10000010$ .

IEEE 754 Standard Representation:  $010000010010110000000000000000$

1 Sign + 8 Exponent + 23 Mantissa = 32 bits

# Representing Text





# STANDARDS: ASCII vs UNICODE



| ASCII                                     | Unicode                                                                 | Extended ASCII           |
|-------------------------------------------|-------------------------------------------------------------------------|--------------------------|
| 7 bits per character                      | 16, 24, or 32 bits per character                                        | 8 bits per character     |
| 128 different characters                  | Between 65,536 – 4,294,967,296 characters depending on the system ASCII | 256 different characters |
| All English letters, numbers, and symbols | Contains all major languages, symbols, hieroglyphics, etc.              |                          |

# ASCII Code

| Decimal | Hex | Char                   | Decimal | Hex | Char    | Decimal | Hex | Char | Decimal | Hex | Char  |
|---------|-----|------------------------|---------|-----|---------|---------|-----|------|---------|-----|-------|
| 0       | 0   | [NULL]                 | 32      | 20  | [SPACE] | 64      | 40  | @    | 96      | 60  | `     |
| 1       | 1   | [START OF HEADING]     | 33      | 21  | !       | 65      | 41  | A    | 97      | 61  | a     |
| 2       | 2   | [START OF TEXT]        | 34      | 22  | "       | 66      | 42  | B    | 98      | 62  | b     |
| 3       | 3   | [END OF TEXT]          | 35      | 23  | #       | 67      | 43  | C    | 99      | 63  | c     |
| 4       | 4   | [END OF TRANSMISSION]  | 36      | 24  | \$      | 68      | 44  | D    | 100     | 64  | d     |
| 5       | 5   | [ENQUIRY]              | 37      | 25  | %       | 69      | 45  | E    | 101     | 65  | e     |
| 6       | 6   | [ACKNOWLEDGE]          | 38      | 26  | &       | 70      | 46  | F    | 102     | 66  | f     |
| 7       | 7   | [BELL]                 | 39      | 27  | '       | 71      | 47  | G    | 103     | 67  | g     |
| 8       | 8   | [BACKSPACE]            | 40      | 28  | (       | 72      | 48  | H    | 104     | 68  | h     |
| 9       | 9   | [HORIZONTAL TAB]       | 41      | 29  | )       | 73      | 49  | I    | 105     | 69  | i     |
| 10      | A   | [LINE FEED]            | 42      | 2A  | *       | 74      | 4A  | J    | 106     | 6A  | j     |
| 11      | B   | [VERTICAL TAB]         | 43      | 2B  | +       | 75      | 4B  | K    | 107     | 6B  | k     |
| 12      | C   | [FORM FEED]            | 44      | 2C  | ,       | 76      | 4C  | L    | 108     | 6C  | l     |
| 13      | D   | [CARRIAGE RETURN]      | 45      | 2D  | -       | 77      | 4D  | M    | 109     | 6D  | m     |
| 14      | E   | [SHIFT OUT]            | 46      | 2E  | .       | 78      | 4E  | N    | 110     | 6E  | n     |
| 15      | F   | [SHIFT IN]             | 47      | 2F  | /       | 79      | 4F  | O    | 111     | 6F  | o     |
| 16      | 10  | [DATA LINK ESCAPE]     | 48      | 30  | 0       | 80      | 50  | P    | 112     | 70  | p     |
| 17      | 11  | [DEVICE CONTROL 1]     | 49      | 31  | 1       | 81      | 51  | Q    | 113     | 71  | q     |
| 18      | 12  | [DEVICE CONTROL 2]     | 50      | 32  | 2       | 82      | 52  | R    | 114     | 72  | r     |
| 19      | 13  | [DEVICE CONTROL 3]     | 51      | 33  | 3       | 83      | 53  | S    | 115     | 73  | s     |
| 20      | 14  | [DEVICE CONTROL 4]     | 52      | 34  | 4       | 84      | 54  | T    | 116     | 74  | t     |
| 21      | 15  | [NEGATIVE ACKNOWLEDGE] | 53      | 35  | 5       | 85      | 55  | U    | 117     | 75  | u     |
| 22      | 16  | [SYNCHRONOUS IDLE]     | 54      | 36  | 6       | 86      | 56  | V    | 118     | 76  | v     |
| 23      | 17  | [ENG OF TRANS. BLOCK]  | 55      | 37  | 7       | 87      | 57  | W    | 119     | 77  | w     |
| 24      | 18  | [CANCEL]               | 56      | 38  | 8       | 88      | 58  | X    | 120     | 78  | x     |
| 25      | 19  | [END OF MEDIUM]        | 57      | 39  | 9       | 89      | 59  | Y    | 121     | 79  | y     |
| 26      | 1A  | [SUBSTITUTE]           | 58      | 3A  | :       | 90      | 5A  | Z    | 122     | 7A  | z     |
| 27      | 1B  | [ESCAPE]               | 59      | 3B  | ;       | 91      | 5B  | [    | 123     | 7B  | {     |
| 28      | 1C  | [FILE SEPARATOR]       | 60      | 3C  | <       | 92      | 5C  | \    | 124     | 7C  |       |
| 29      | 1D  | [GROUP SEPARATOR]      | 61      | 3D  | =       | 93      | 5D  | ]    | 125     | 7D  | }     |
| 30      | 1E  | [RECORD SEPARATOR]     | 62      | 3E  | >       | 94      | 5E  | ^    | 126     | 7E  | ~     |
| 31      | 1F  | [UNIT SEPARATOR]       | 63      | 3F  | ?       | 95      | 5F  | _    | 127     | 7F  | [DEL] |

Sample program: <https://onlinegdb.com/zSG4LNMu4>

# ASCII Applications

- Used for numerals
  - Social Security numbers
  - Phone numbers
- Plain, unformatted text (ASCII text) stored in text file (.txt)
  - Apple: “Plain Text”
  - Windows: “Text Document”



# Formatted Text Files

- To create documents with styles and formats, formatting codes must be embedded in the text.
  - Microsoft Word (DOCX)
  - Apple Pages (PAGES)
  - Adobe Acrobat (PDF)
  - Etc.





Coasters.DOCX



Here is the document title. The formatting codes that make it bold are delimited by < and > angle brackets.

xmlns:wps="http://schemas.microsoft.com/office/word/2010/wordprocessingShape" mc:Ignorable="w14 w15 wp14"><w:body><w:p><w:r><w:t><w:strong>Roller Coasters</w:strong></w:t></w:r></w:p><w:p w:rsidR="002107BB" w:rsidRDefault="003E2449"><w:r w:rsidRPr="003E2449"><w:rPr><w:i/></w:rPr><w:t>Who wants to save an old roller coaster?</w:t></w:r><w:r w:rsidRPr="003E2449"><w:t xml:space="preserve"> Leap-the-Dips is the world's oldest roller coaster and, according to a spokesperson for the Leap-the-Dips Preservation Foundation, one of the most historically significant. Built in 1902, Leap-the-Dips</w:t></w:r><w:proofErr w:type="gramStart"/><w:r w:rsidRPr="003E2449"><w:t>is</w:t></w:r><w:proofErr w:type="gramEnd"/><w:r w:rsidRPr="003E2449"><w:t xml:space="preserve"> "the sole survivor of a style and technology that was represented in more than 250 parks in North America alone in the early years of the amusement industry.</w:t></w:r><w:r w:rsidR="00881463"><w:t>"

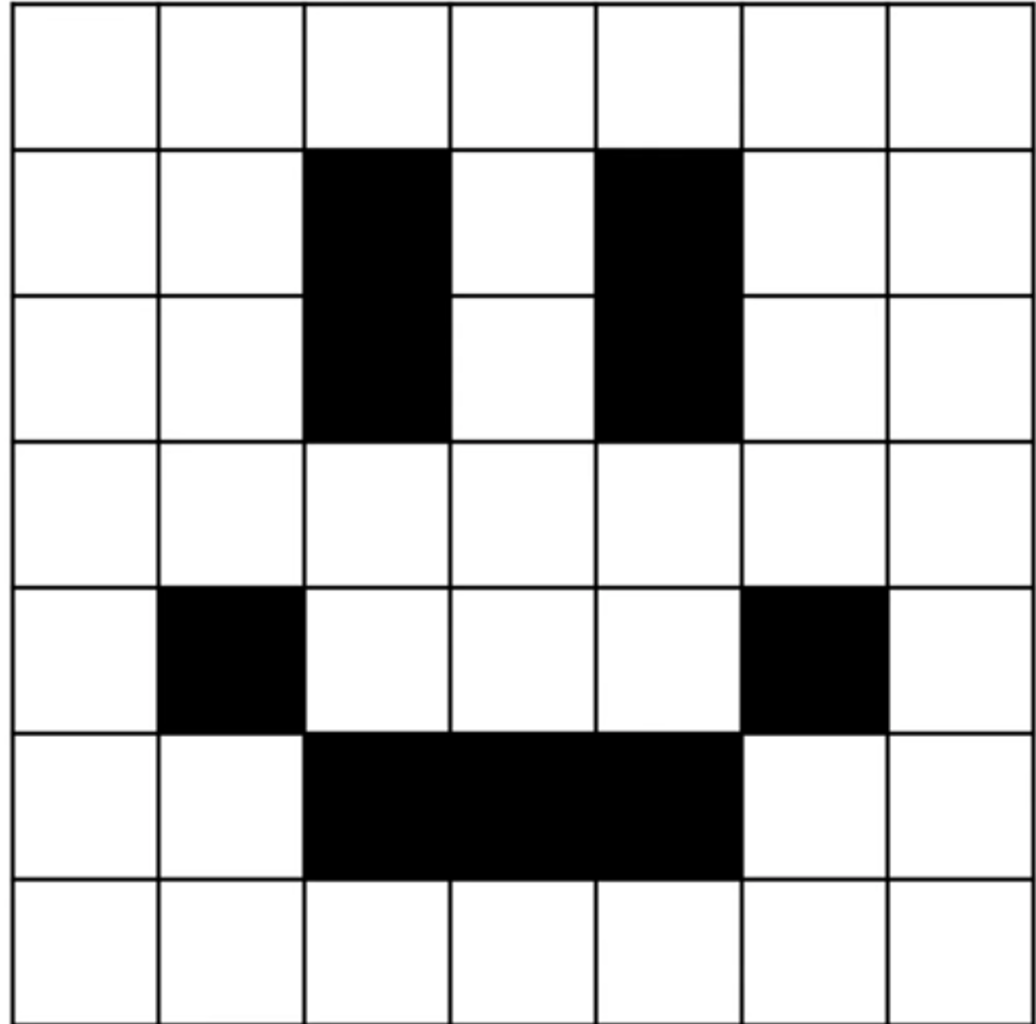


# Representing Images and Graphics



# Images

- How could we use binary to represent this image?
- How many colors we need?

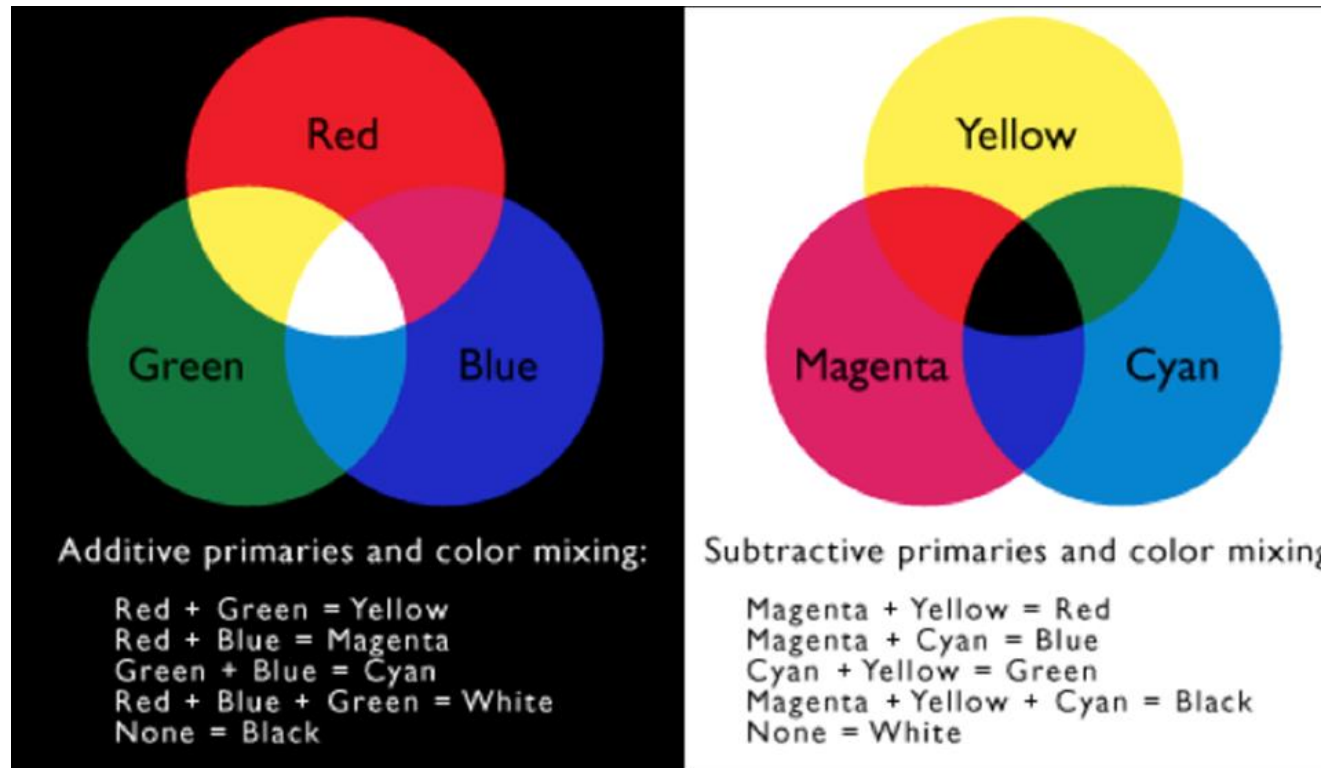


# Images

- 2 colors: 1 for white, 0 for black
- A **pixel** (short for "picture element") is the smallest unit of a digital image or display that can be individually manipulated.
- Pixels are the building blocks of any digital image, and each pixel represents a single point of color in the image.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 |

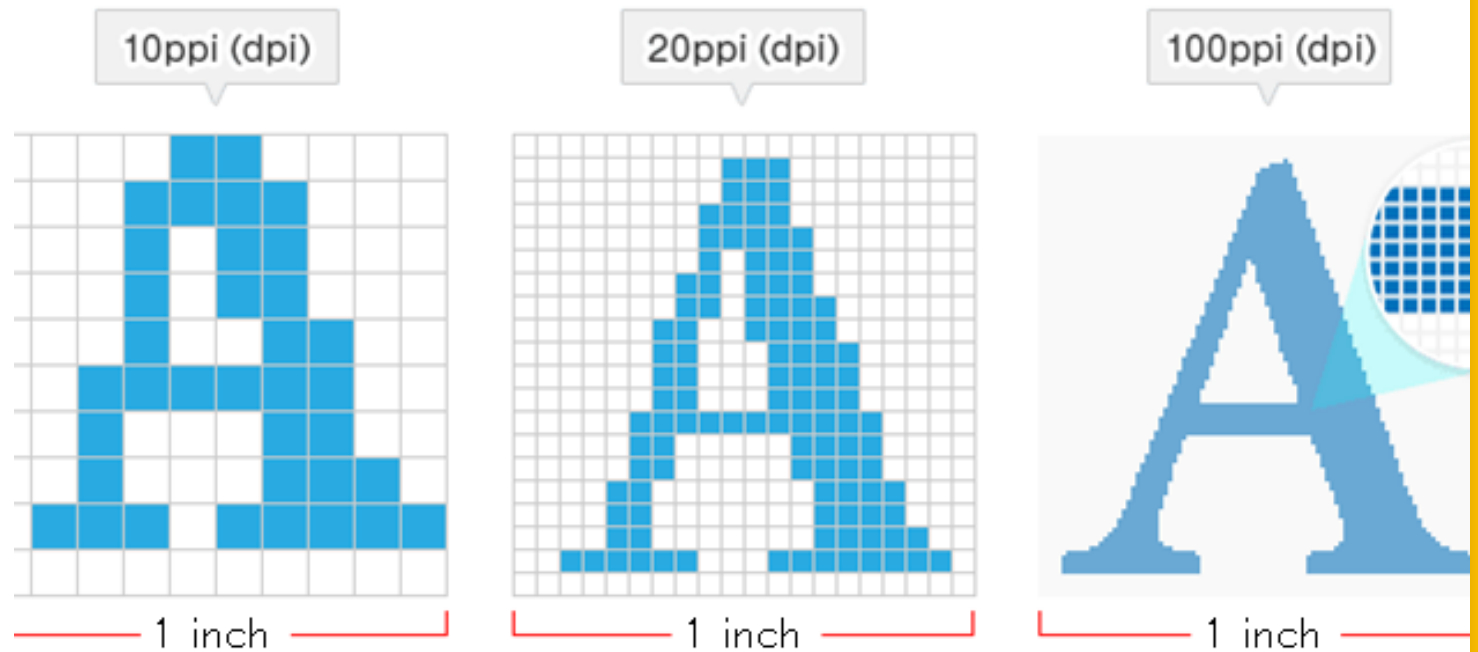
# Color



- Color is expressed as a combination of three colors of light
- Each color value is represented as one byte, so each color value ranges from 0 to 255
- Two Types:
  - CMY (cyan-magenta-yellow) → Printers
  - RGB (red-green-blue) → Screen

# Resolution

- The **resolution** of an image refers to the number of pixels it contains.
- Typically expressed as width × height (e.g., 1920 × 1080).
- More pixels mean higher resolution and more detail.

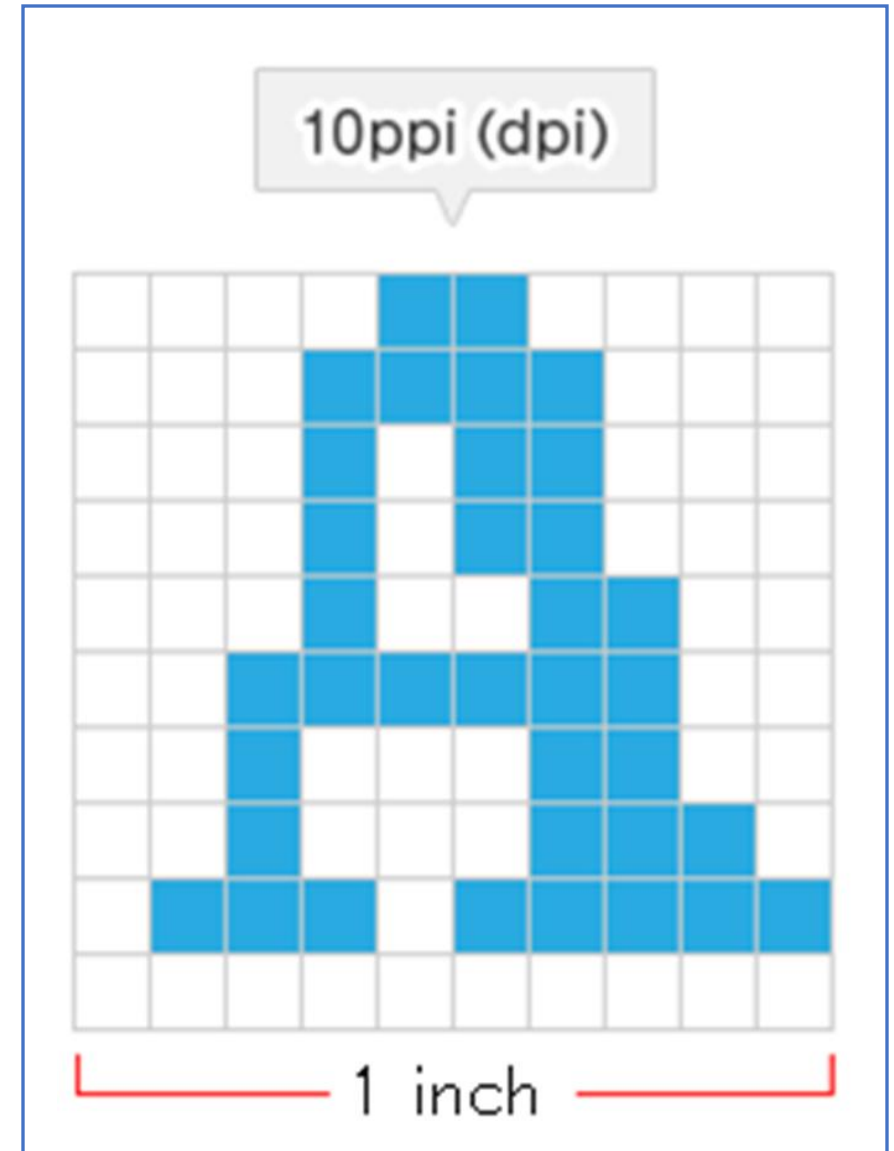


# Storage

- Image Specifications:
  - Resolution: 10ppi (10p x 10p)
  - RGB: 3 bytes

How many bytes would be needed to store this image?

- $10p \times 10p = 100$  pixels
- Each pixel has 3 bytes  $\rightarrow 100 * 3 = 300$  bytes



# Medical Imaging

- **Magnetic Resonance Imaging (MRI)**: non-invasive imaging technique that uses strong magnetic fields and radio waves to generate detailed images of **soft tissues**, organs, and other internal structures.
- **Positron Emission Tomography (PET)**: allows for the observation of **metabolic processes** in the body.
- **Computed Tomography (CT scans)**: use X-rays to create **cross-sectional images** of the body.
- **Ultrasound**: Uses high-frequency **sound waves** to create real-time images of the body's internal structures.
- **X-rays**: used to view bones, detect fractures, and identify lung conditions.

# Medical Imaging

- Each **pixel** or **voxel** in the image corresponds to a specific part of the anatomy, with colors and intensities reflecting different tissues and structures.
- A **voxel** (short for "volume element") is the three-dimensional equivalent of a pixel.
- While a **pixel** represents a single point in a 2D image, a voxel represents a point in 3D space

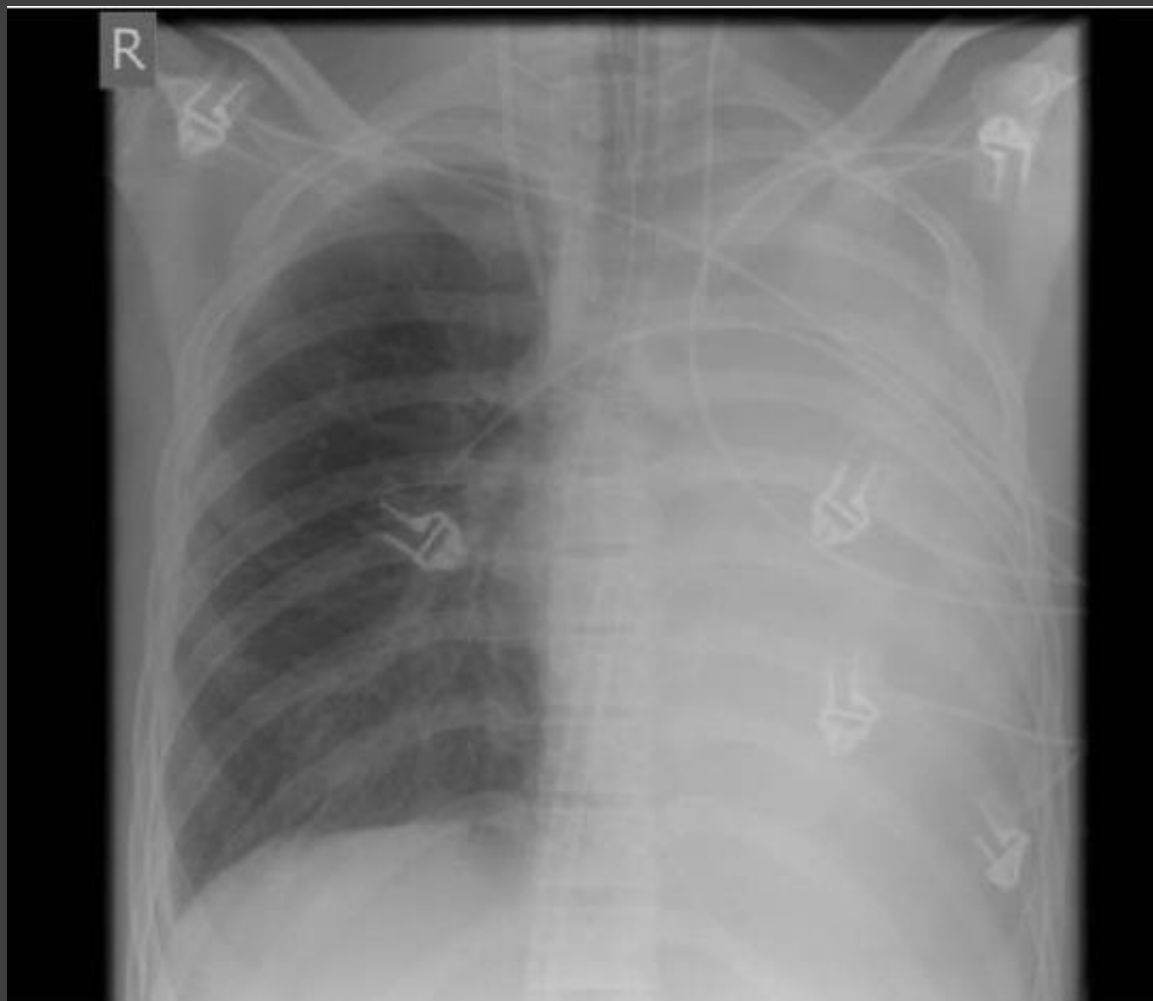


# CT Scan



# MRI Scan

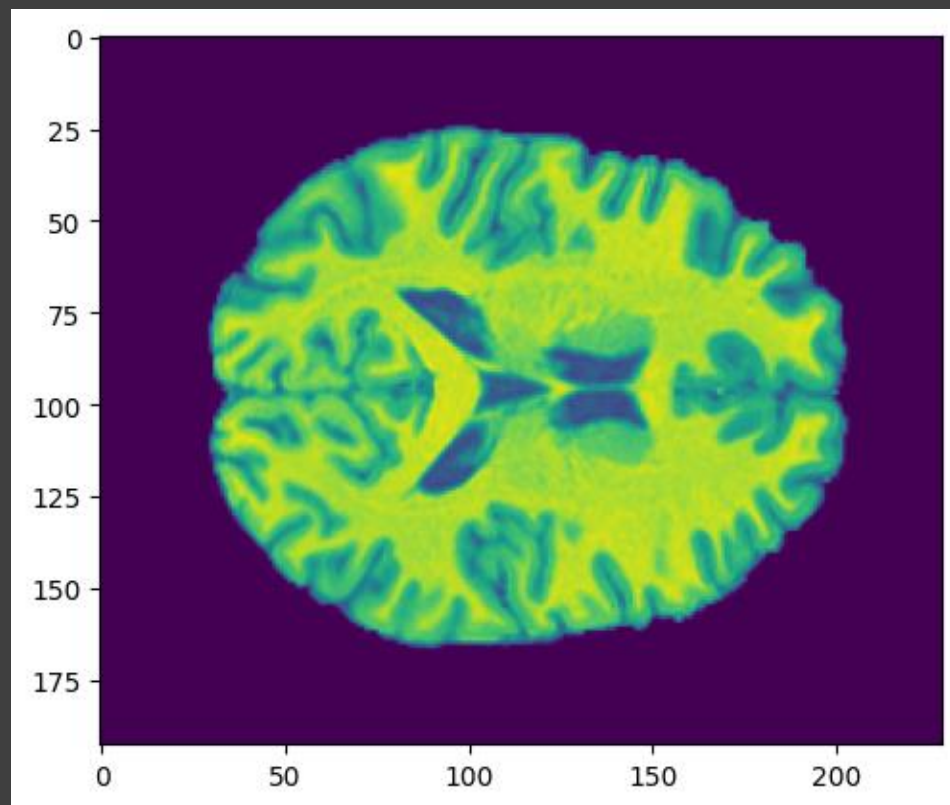




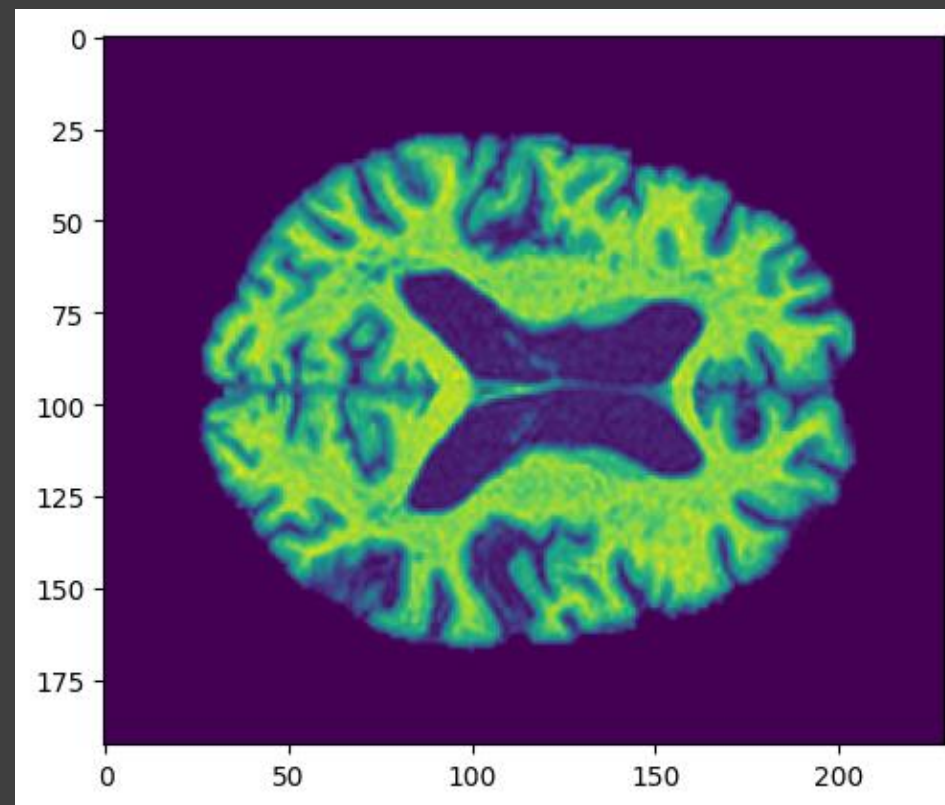
X-Rays



Ultrasound

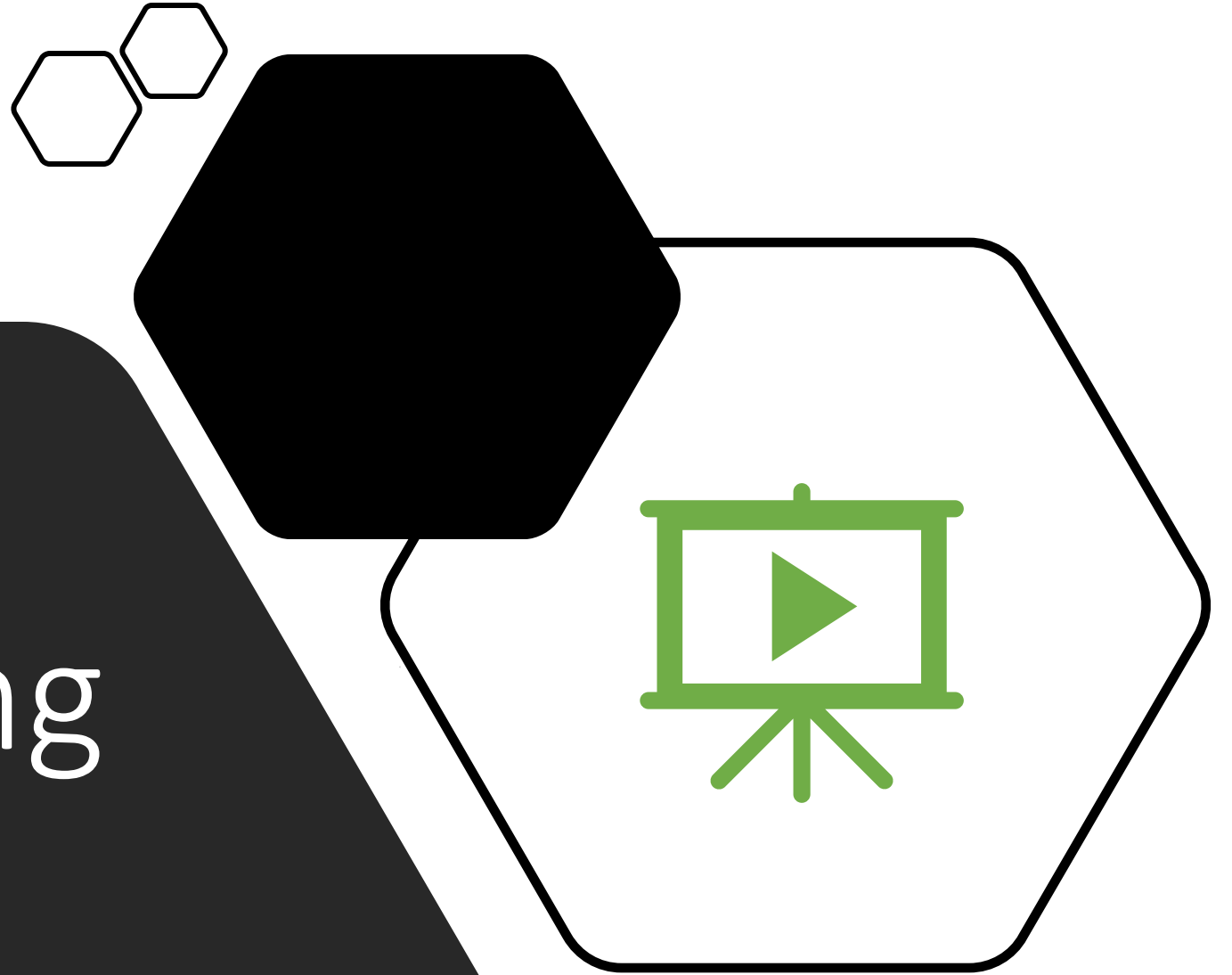


Normal Aging



Alzheimer's Disease

# Representing Video



# What Is A Video?

---

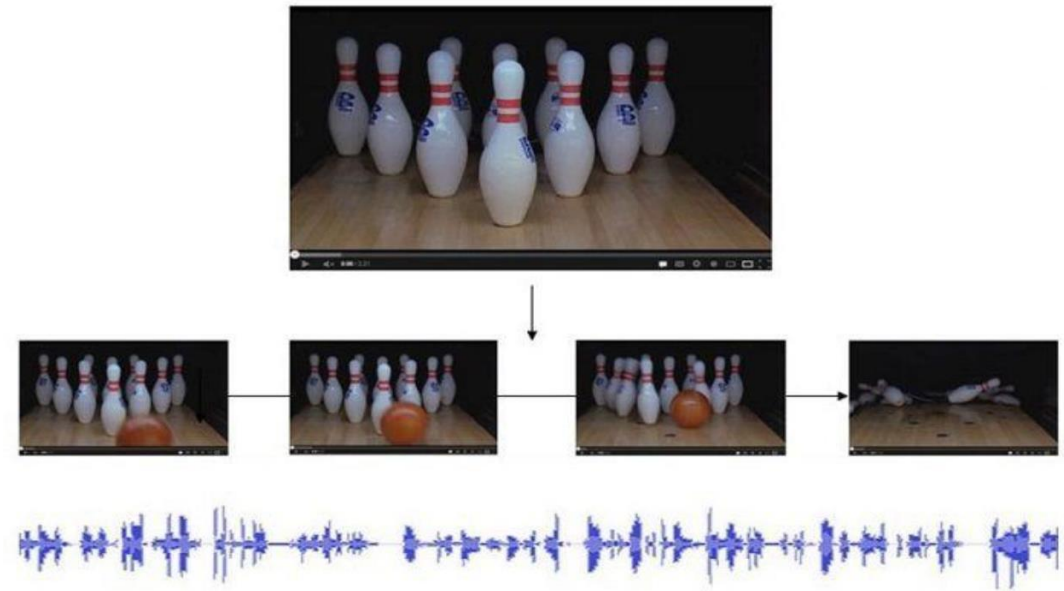
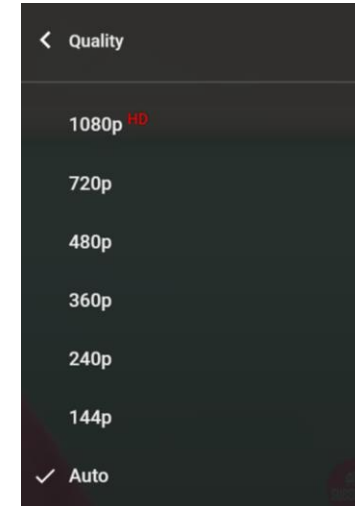
- Understood as a series of pictures being “flipped” through with audio
- The same/similar concepts for image and audio representation apply to video





# Digital Video

- Made up of individual **frames** (images), each one representing a time slice of the scene with audio stored as a separate stream
- **Frame Rate**: frequency at which frames are displayed
- Video **Quality**:
  - The higher the frame rate the smoother the video will appear
  - The higher resolution the higher the quality of the image

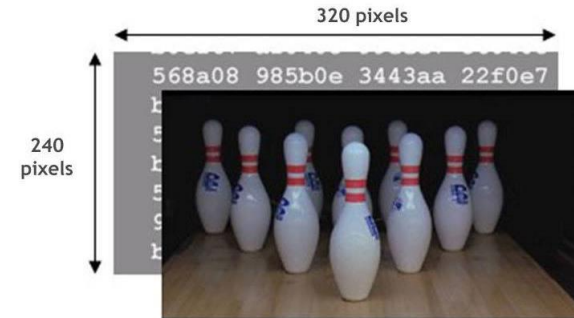


# Storage - Example

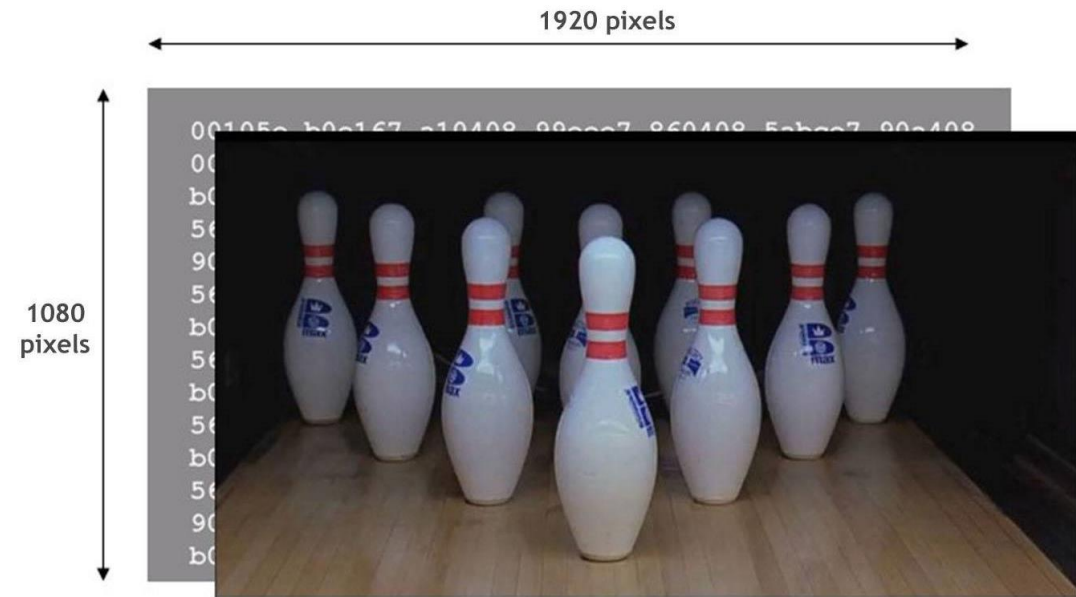
- Video Specifications:
  - Resolution: (Low Resolution) 240p x 320p
  - Frame Rate: 24 frames per second

How many bytes would be needed to store 1 minute of video?

- $240 \times 320 = 76,800$  pixels in single frame
- Each pixel 3 bytes (RGB)  $\rightarrow$  230,400 bytes or about 200 KB in a single frame
- 1 second of video  $\rightarrow$  5,529,600 bytes or 5 MB
- 1 minute of video  $\rightarrow$  331,776,000 bytes or 332 MB



Low resolution



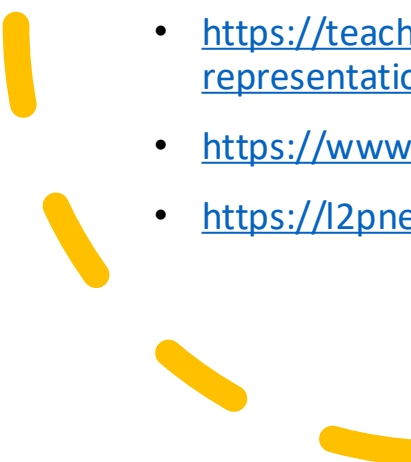
HD resolution



# References



- <https://home.adelphi.edu/~siegfried/cs170/170l1.pdf>
- <https://csfieldguide.org.nz/en/chapters/data-representation/whats-the-big-picture/>
- <https://www.digitaltechnologieshub.edu.au/teachers/topics/data-representation>
- <https://i.pinimg.com/564x/02/0f/30/020f300815ddf2473909c8068fa933cd.jpg>
- <https://i.ytimg.com/vi/4M6L-ubixJo/maxresdefault.jpg>
- [https://www.cplusplus.com/doc/hex/base\\_hexadecimal.gif](https://www.cplusplus.com/doc/hex/base_hexadecimal.gif)
- [https://usercontent2.hubstatic.com/14086459\\_f520.jpg](https://usercontent2.hubstatic.com/14086459_f520.jpg)
- [https://cdn.ttgtmedia.com/rms/onlineImages/storage-memory\\_capacity\\_chart\\_mobile.png](https://cdn.ttgtmedia.com/rms/onlineImages/storage-memory_capacity_chart_mobile.png)
- [https://i.ytimg.com/vi/TpoV\\_iVhFt0/maxresdefault.jpg](https://i.ytimg.com/vi/TpoV_iVhFt0/maxresdefault.jpg)
- <https://sites.temple.edu/lizhe/2018/09/16/introduce-the-world-of-ascii-and-unicode/#:~:text=Ascii%20stands%20for%20American%20Standard,Unicode%20defines%202%5E21%20characters.>
- <https://static.thenounproject.com/png/56875-200.png>
- <https://image.flaticon.com/icons/png/512/32/32745.png>

- 
- 
- <https://app.teamsupport.com/dc/493740/images/c698427c-c702-4463-893f-59d63ab14233.png>
  - <https://image.flaticon.com/icons/png/512/29/29072.png>
  - <https://www.logaster.com/blog/vector-and-raster-graphics/>
  - <https://learn.sparkfun.com/tutorials/analog-vs-digital/all>
  - <https://www.djtechdirect.com/blog/wp-content/uploads/2018/07/Digital-and-Analog-Wave.jpg>
  - [https://lh3.googleusercontent.com/proxy/HxkV77Agc9AniouomafNZ37L2Msc8GDzv9rS\\_FXRelt\\_N-rHcriAI954kxEHZKnZuSoxobqNZFzekwEB8CC3-o1nAd0dwPSujNuF6f\\_kWZUBPBYIt58AzjZFUevwa7ZtJsgjkQ0](https://lh3.googleusercontent.com/proxy/HxkV77Agc9AniouomafNZ37L2Msc8GDzv9rS_FXRelt_N-rHcriAI954kxEHZKnZuSoxobqNZFzekwEB8CC3-o1nAd0dwPSujNuF6f_kWZUBPBYIt58AzjZFUevwa7ZtJsgjkQ0)
  - [https://mynewmicrophone.com/wp-content/uploads/2019/08/mnm\\_How\\_Do\\_Microphone\\_Transducers\\_Work.jpg](https://mynewmicrophone.com/wp-content/uploads/2019/08/mnm_How_Do_Microphone_Transducers_Work.jpg)
  - <https://mynewmicrophone.com/are-microphones-analog-or-digital/>
  - <https://teachcomputerscience.com/sound-representation/#:~:text=How%20can%20sound%20be%20sampled,converted%20into%20a%20binary%20number.>
  - <https://www.youtube.com/watch?v=iqjzZDWoW7s>
  - <https://l2pnet.com/digital-recording-does-not-chop-your-music-11102010/>